OTA Compliance

Price discovery, monitoring, and review

compliance.osakiplus.com

I built OTA Compliance to make advertised-price review a repeatable process. The application finds relevant product listings, checks prices against minimum advertised price (MAP) references, and brings the latest results and source context into one dashboard. Its scope is the selected product models, configured dealer websites, and additional pages found through external search.

Keeping up with changing listings

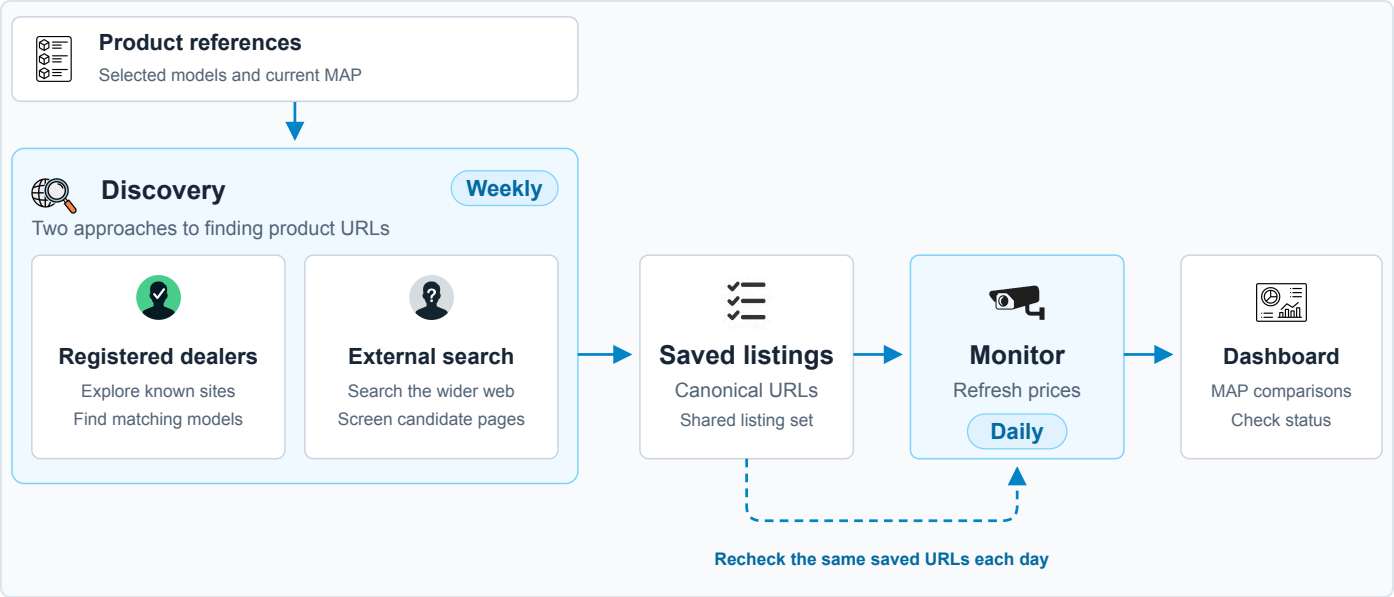
Reviewing advertised prices by hand means finding the right product page, identifying the model, comparing the price with the current MAP, and keeping enough context to revisit it later. That work repeats across websites and over time. Product aliases, different page layouts, and changing URLs make the comparison harder to maintain consistently.

A useful result also needs a source and a capture time. A page may change, become unavailable, or block a request. I designed the project to keep those outcomes visible alongside successful observations, so reviewers can see both the available pricing evidence and the gaps that still need follow-up.

Finding pages and checking them again

The workflow separates finding product pages from rechecking their prices. Discovery contains two approaches, registered dealer discovery and external search, which feed one saved listing set.

Discovery expands the listing set weekly; Monitor refreshes saved price observations daily.



Registered dealer discovery and external search share one listing set for daily monitoring.

Two different discovery problems

Registered dealer discovery addresses coverage on known websites. The system knows which dealers to inspect, but still has to find the right product pages among different catalogs and page structures. It uses available feeds, collections, site searches, and sitemaps to locate the selected models within those sites.

External search looks for additional listings across the wider web, including sites outside the registered dealer list. It uses Serper to retrieve structured Google Search and Shopping results for product names and aliases. Returned titles and URLs are screened for unrelated content, ambiguous model matches, and duplicates before candidate pages are checked.

Separating the approaches lets each use the right search method and request limits.

Monitor: daily price checks

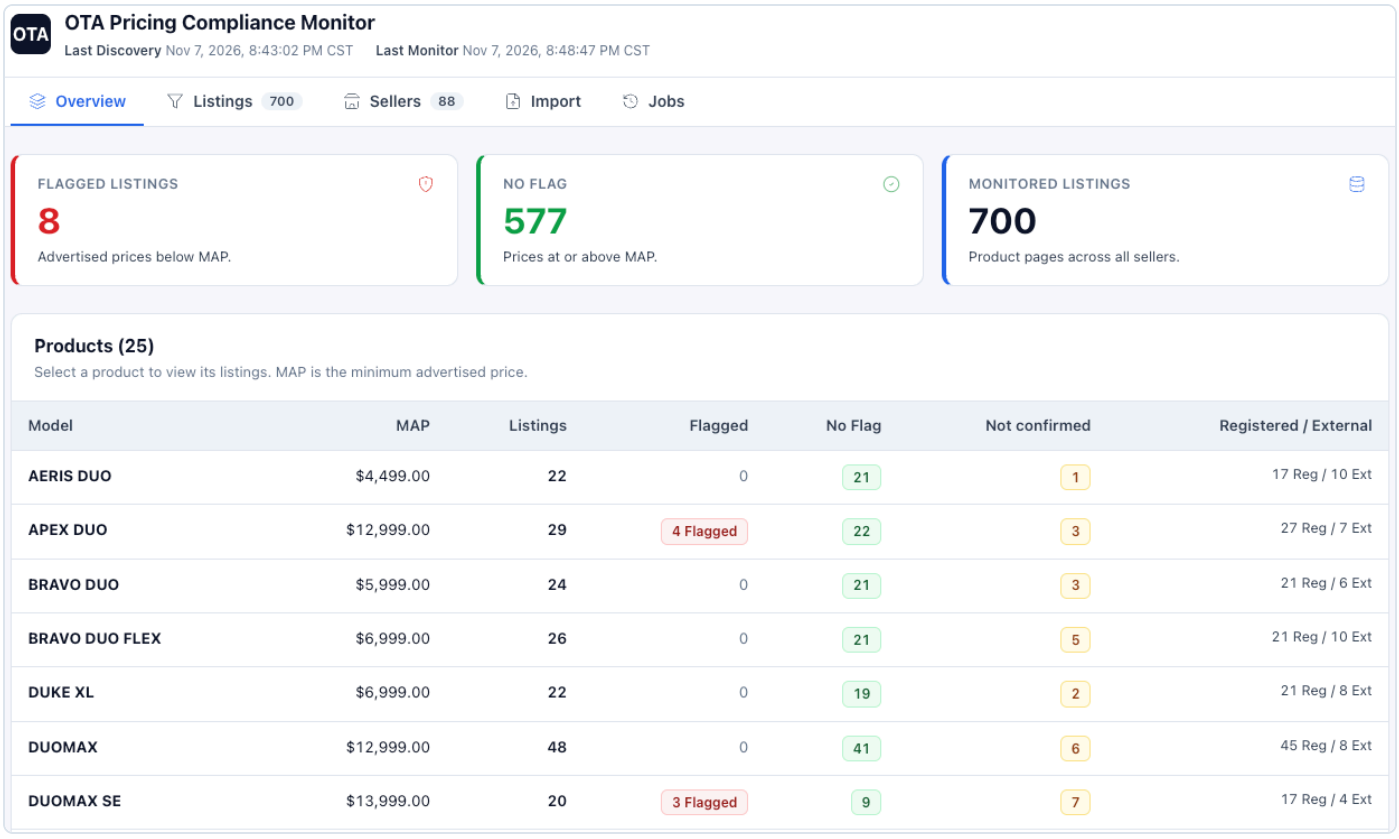
Monitor runs daily against eligible saved URLs. It confirms the product model and extracts the advertised price, currency, and availability from product pages or public catalog data. Each attempt saves its capture time and supporting evidence while preserving earlier checks.

If a page rejects access or its price cannot be confirmed, the website outcome and available source context remain visible. These listings stay unresolved for review.

Reviewing the latest results

The dashboard groups listings by model and seller and shows each latest check's price, MAP comparison, flag, and capture time. Listing details provide the check status and available source context.

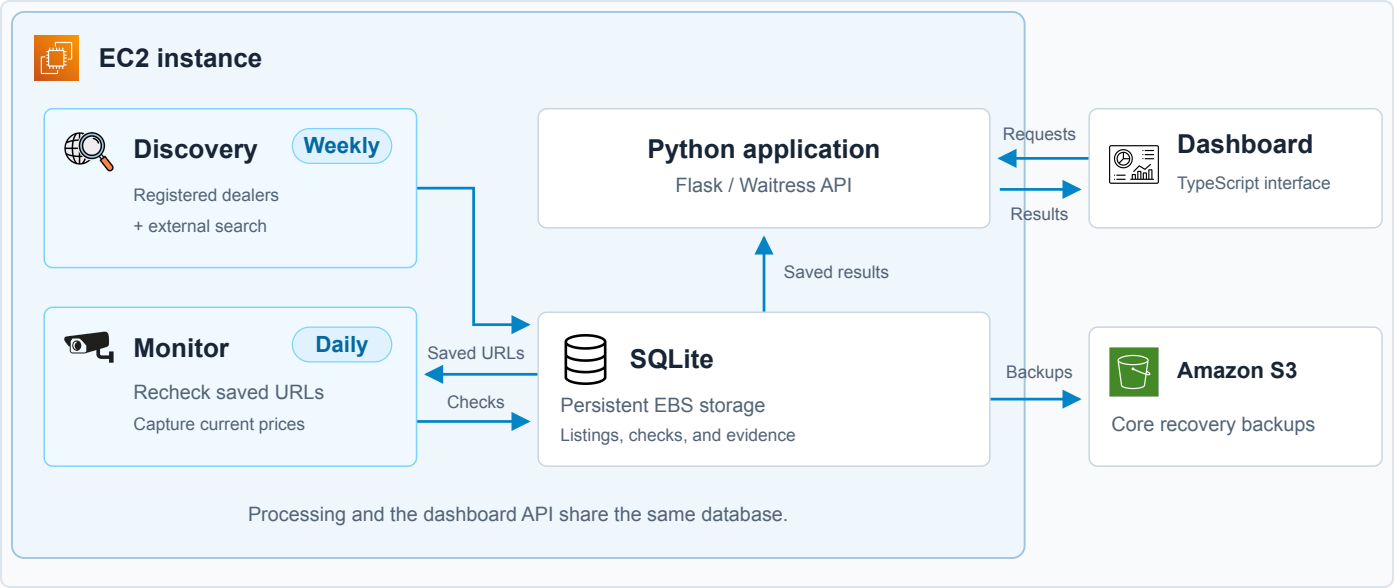
Earlier checks remain stored in SQLite with their original timestamps, MAP values, and captured facts.



The dashboard summarizes monitored listings and highlights product models with flagged prices or checks that still need confirmation.

Running the service

The TypeScript dashboard is backed by a Python application using Flask and Waitress on an EC2 instance. Discovery and Monitor store their results in SQLite on persistent EBS storage, alongside captured evidence and run summaries. A private S3 bucket stores recovery bundles for the database and core configuration.



Persistent EBS storage keeps operational data; S3 holds core recovery backups.

PROJECT CASE STUDY

OTA Support

Technician Resource Portal

I built otasupport.com to help technicians find service manuals, technical documents, and other reference materials independently, while reducing routine access requests to the warranty department.

Why the project was needed

The resource library already existed in Google Drive, but technicians relied on the warranty department to arrange access. Staff handled permissions manually alongside their warranty responsibilities, and technicians depended on that intervention before they could use the materials.

Finding information was another difficulty. A shared folder provided access to the files, but navigating a large library through Google Drive was cumbersome. The project needed to improve both access to published resources and document discovery.

What I built

The portal presents the library through search and folder navigation. Technicians can search document names and folder paths or browse the existing hierarchy. Selecting a resource opens it directly in Google Drive, which continues to handle file access and viewing.

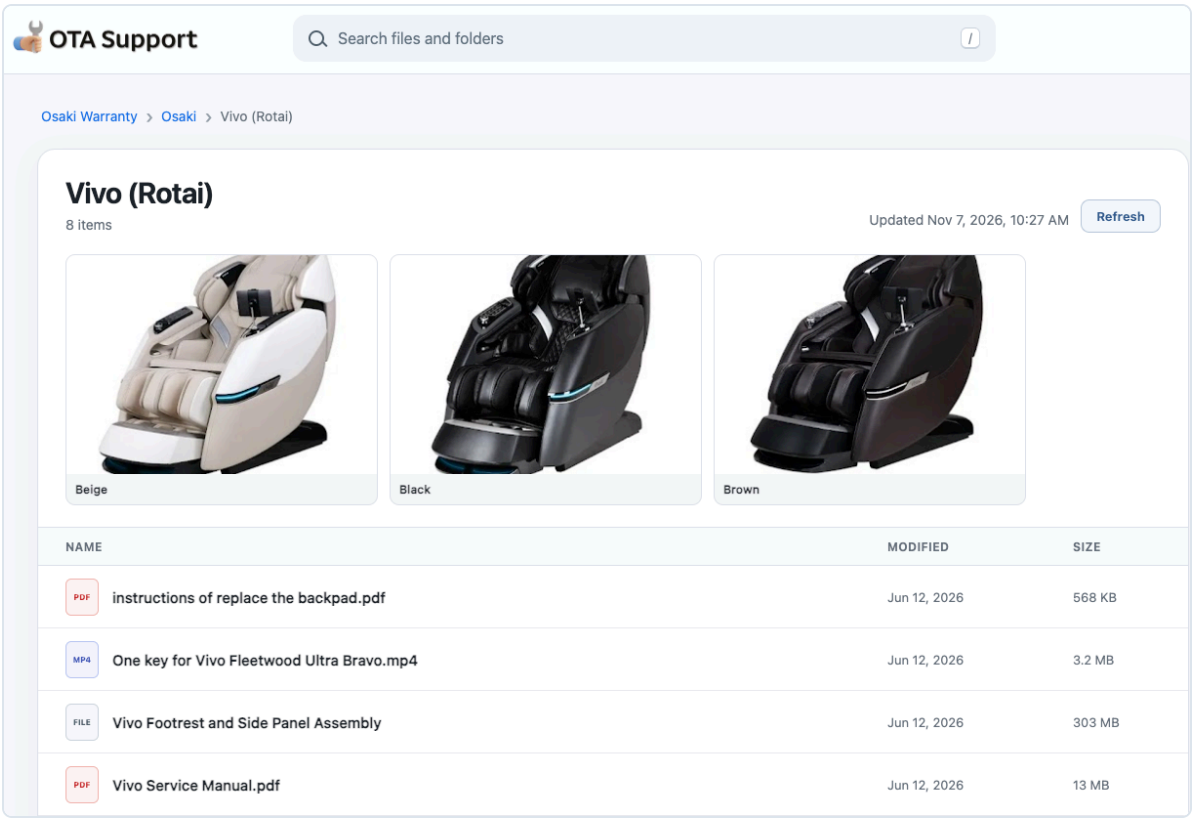


Figure 1. Technicians can search the library, browse its folders, and open a manual or other service resource.

Publishing and architecture

Keeping the library current

I designed the publishing workflow around the tools staff already used. Google Drive remains the source library: staff organize and update documents in familiar folders, and the portal builds its catalog from that structure. This keeps the website aligned with the library without requiring a second collection of documents or a separate publishing system.

Every folder named **In Production (Hold)** and its contents are excluded from the portal's catalog. This gives staff a designated place for unfinished material that should not appear in search or navigation. Google Drive sharing settings continue to govern access to the files.

How the application works

The application is written in TypeScript. A Node.js web application runs on an EC2 instance, S3 stores the prepared catalog, and a Lambda function reads Google Drive to generate it. EventBridge Scheduler runs the scan daily; the portal's Refresh action can invoke the same process when a change needs to appear sooner.

The catalog contains document names, folder relationships, and links to the original files. Manuals, videos, and other resources remain in Google Drive, which provides document viewing and media delivery.

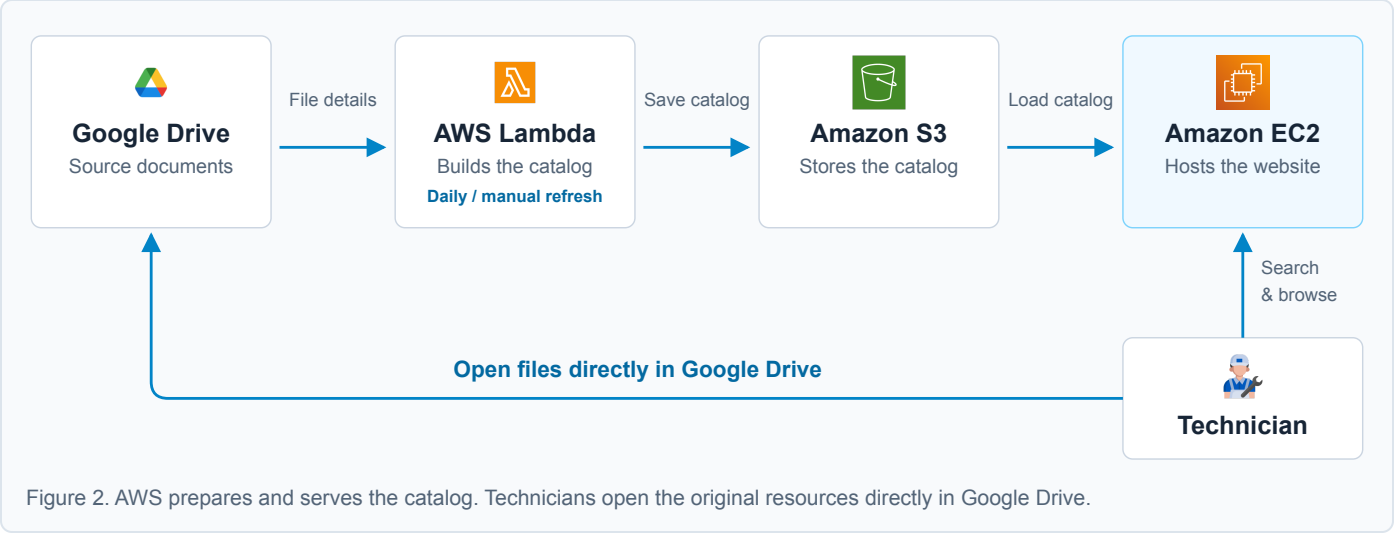


Figure 2. AWS prepares and serves the catalog. Technicians open the original resources directly in Google Drive.

Result

The earlier version recorded **4,573 visitors** and **13,200 page views** over a 12-month period. The portal gives technicians a central place to locate published resources independently, while staff retain a familiar process for keeping the library current.

Capstone Project:
Plant Disease Detection

Role: AI Engineer
Core Technologies: Computer Vision, Multilabel-Classification, CycleGAN

Project Overview[1]

Our team developed a plant pathology diagnosis application to detect and classify Korean crop diseases from a leaf image. The final production model successfully classifies 72 distinct classes, covering 50 different diseases and normal states across 22 types of agricultural crops.

Data Curation & Final Performance

Our team scaled the initially provided dataset (3 crops, 8 diseases) into a massive, robust dataset by sourcing additional data from AI HUB, Kaggle, and academic researchers.

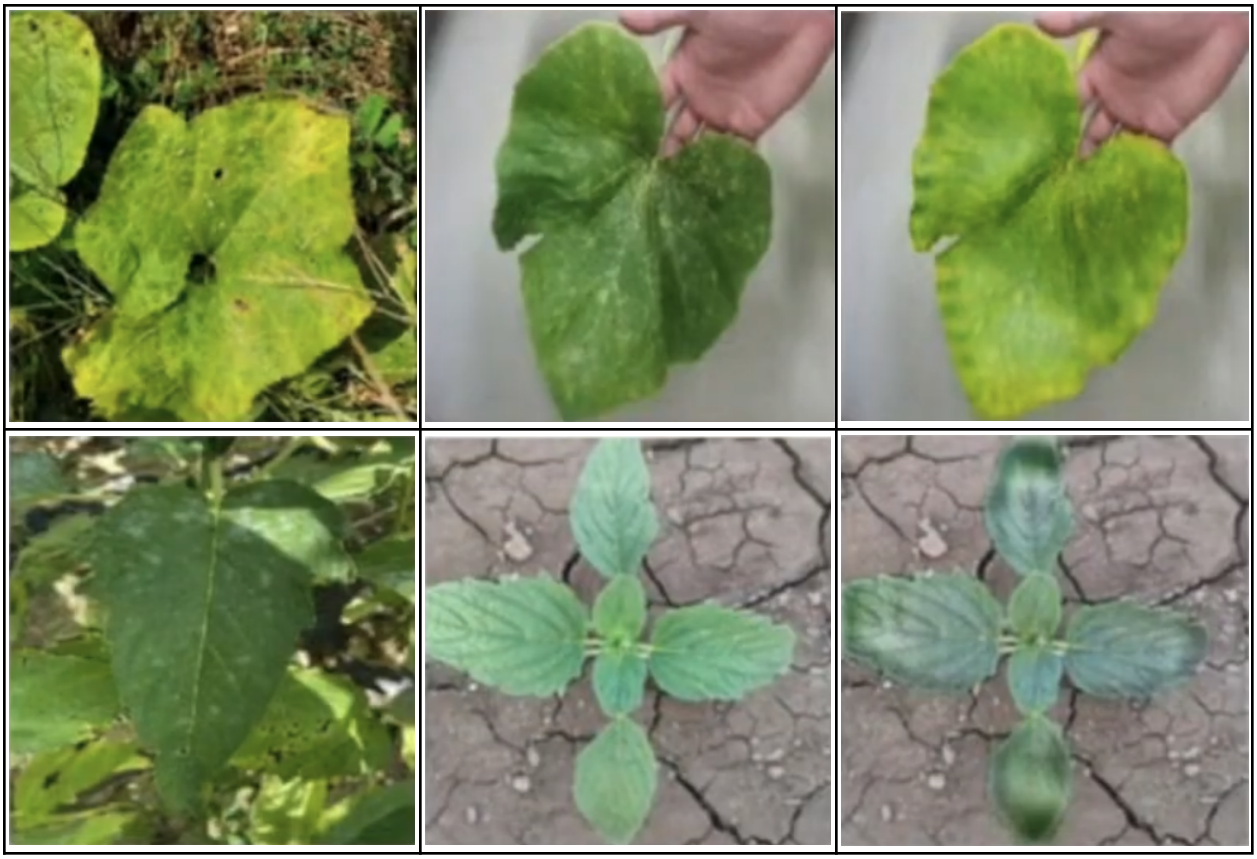
- **Total Dataset:** Curated a final dataset consisting of 33,091 training images and 11,044 evaluation images.
- **Metric Selection (Macro-F1):** Due to the significant class imbalance across the 72 categories (e.g., 2,268 training samples for Red Bean Powdery Mildew vs. only 113 for Sesame Powdery Mildew), the **Macro F1-score** was chosen as the primary evaluation metric. By calculating the metric independently for each class and averaging them equally, this ensures the model's performance isn't skewed by majority classes and reliably detects even the rarest diseases.
- **Performance:** Achieved an overall macro average F1-score of 0.971, with near-perfect (1.00) F1-scores on critical crops like Apple, Eggplant, Strawberry, Tomato, and Watermelon.

Key AI Methodologies & Contributions

CycleGAN[2] for Synthetic Data Generation

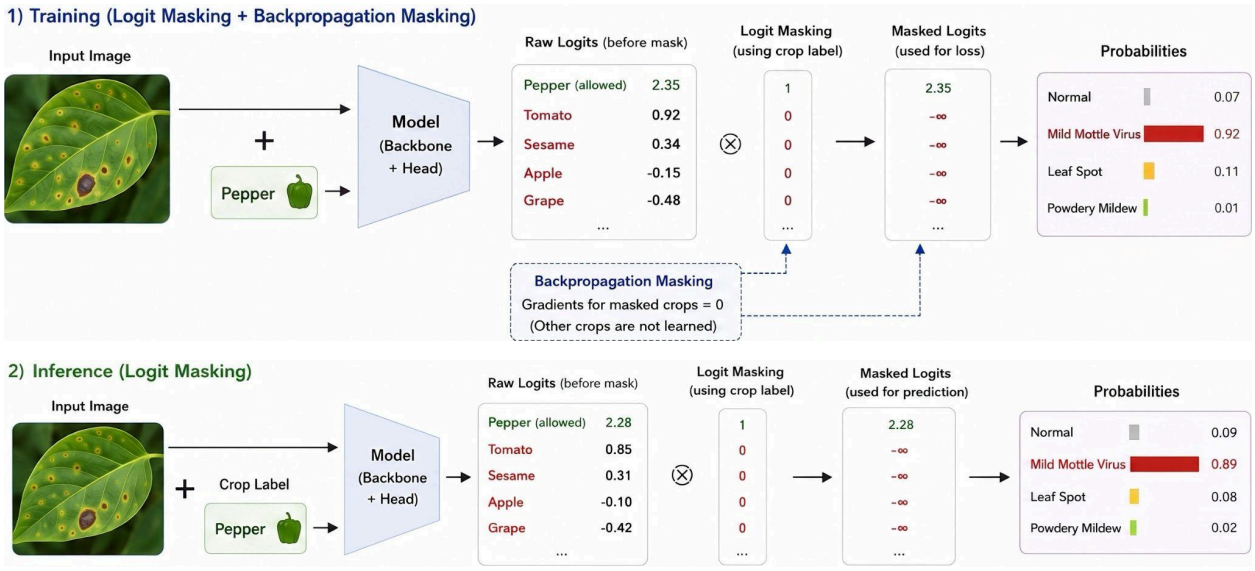
Disease Data	Normal Data	Generated Disease

[1] https://github.com/JeongjunBae/debug-AI/blob/main/README-EN.md
[2] https://arxiv.org/abs/1703.10593



Addressed the high cost and logistical difficulty of acquiring diseased plant images (which typically require isolated environments and expert virus injection). CycleGAN generates realistic diseased images from cheaper, more accessible normal images

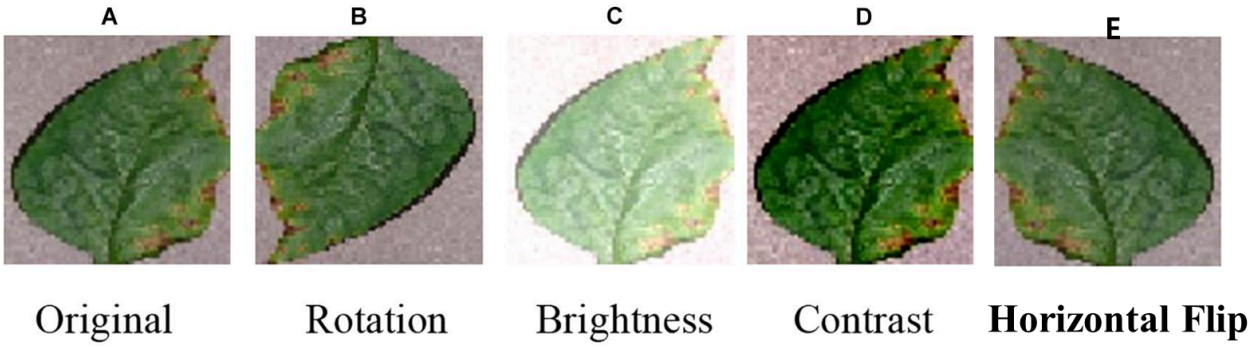
Crop-aware Architecture



Capitalized on the premise that identifying the base crop type is trivial compared to diagnosing the specific disease. Engineered a contextual filtering mechanism that feeds the crop type alongside the image during both training and inference.

- **Training:** During training, backpropagation for non-target crop types is zeroed out.
- **Inference:** During inference, the model masks the confidence scores of irrelevant crop types (setting them to 0), drastically reducing inter-class confusion.

Robust Data Augmentation



Designed a comprehensive augmentation pipeline to further improve model generalization on field data.

Ablation Study & Optimization[3]

Conducted a rigorous ablation study to validate the architectural choices. The baseline model achieved a 0.8729 Validation Macro F1-score. Integrating PlantGAN improved performance to 0.8971, while adding the Crop-aware architecture drove the score up to 0.9655. The final integration of the full augmentation pipeline resulted in the state-of-the-art 0.9710 Validation Macro F1-score.

Ablation Study Table

GAN	Crop-aware	Augmentation	Validation Macro F1-Score
X	X	X	0.8729
O	X	X	0.8971
O	O	X	0.9655
O	O	O	0.9710

Awards

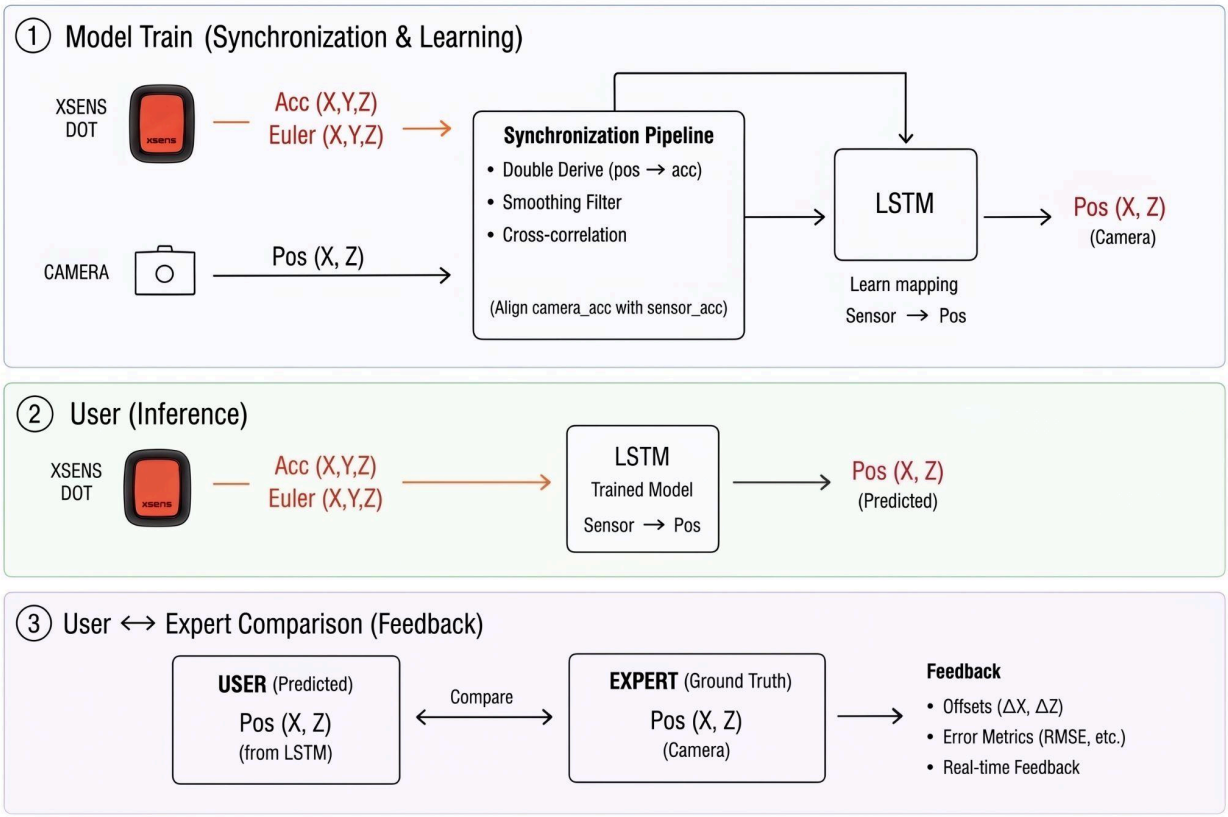
Third Prize[4]: 2022 Sejong University Creative Comprehensive Design Competition.
Popularity Prize[5]: 2022 Sejong University Creative Comprehensive Design Competition.

[3] https://drive.google.com/file/d/1snUUTO9Awm21kvphDfDeMOblA1Oxu7kQ/view?usp=drive_link
[4] https://drive.google.com/file/d/1bahM5ottudMM88nuXQMMx_oCmH7-B7SA/view?usp=drive_link
[5] https://drive.google.com/file/d/1aOuhsl1cmdkzZmD5gbf9-kHjaTDATsRp/view?usp=drive_link

Project Summary: Exercise Correction Using Wearable Devices

Role: Machine Learning Engineer

Project Overview[1]



We developed a sophisticated software and hardware ecosystem to provide real-time exercise correction. The project operates on a three-step machine learning architecture:

1. **Model Train (Synchronization & Learning):** The LSTM model is trained using synchronized data from both an XSENS DOT IMU wearable sensor (acceleration and Euler angles) and a camera (positional data X, Z). The synchronization pipeline aligns the data by double-deriving the camera's position to acceleration, applying a smoothing filter, and using cross-correlation. The camera's positional data serves as the definitive answer labels for the LSTM to learn the sensor-to-position mapping.
2. **User (Inference):** Once the mapping is learned, users perform exercises using only the XSENS DOT wearable sensor. The trained LSTM model takes the real-time sensor data and outputs the predicted position (X, Z) without requiring any camera input.
3. **User ↔ Expert Comparison (Feedback):** The user's predicted positional data is compared directly against an expert's ground truth positional data. By

[1] https://github.com/JeongjunBae/Exercise-correction-using-wearable-devices/blob/main/README-EN.md

calculating positional offsets (ΔX, ΔZ) and error metrics (such as RMSE), the system provides actionable, real-time posture feedback to the user.

For a visual and detailed reference regarding the project's background and development, please refer to the presentation slides[2]. Please note that a band-pass filter may be used for some exercises.

Performance Metrics:

Through this pipeline, the system achieved high precision in tracking, resulting in:

- Vertical Error: **2.5cm**
- Horizontal Error: **4cm**

Background & Motivation

- Although wearable-device-based motion tracking has been actively studied, existing approaches often require numerous sensors, making them impractical for everyday users to wear during exercise.
- The high cost of personal training creates a significant barrier to accessible and proper posture correction.
- Muscle imbalances are highly common. Individuals naturally default to unbalanced and potentially harmful postures when exhausted during intense weight training.

Technical Implementation Overview

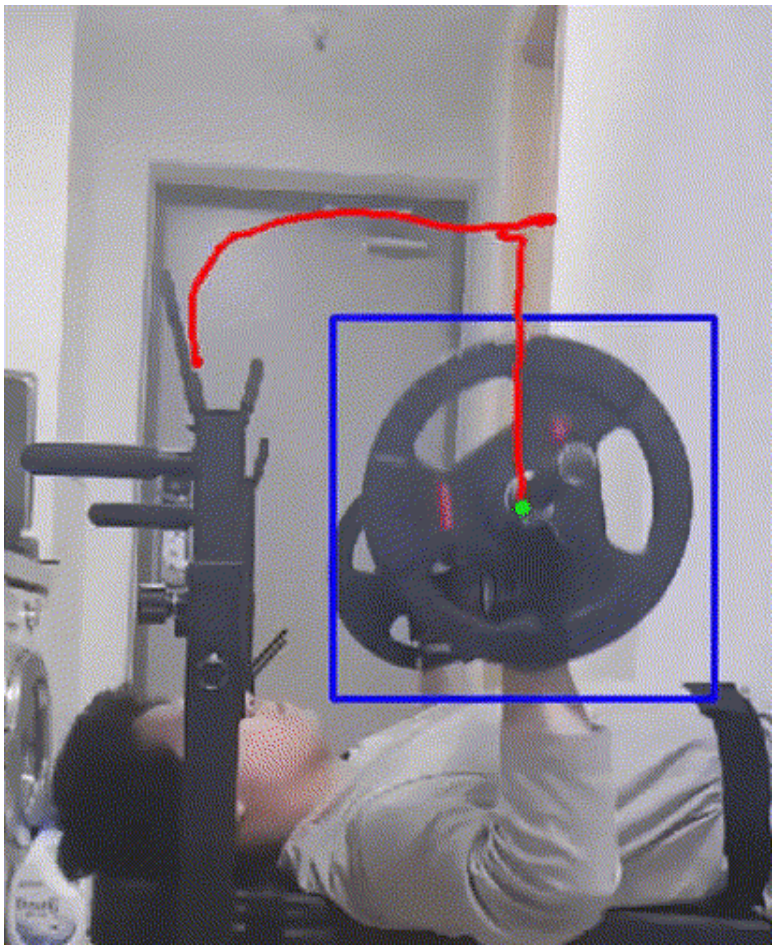
Component	Technology / Approach	Details
IMU Sensors	Xsens Dot	Utilized a highly accurate MEMS sensor for raw data gathering.
Vertical Tracking	Band-pass & High-pass filters	Mitigated integration drift by isolating the vertical position through sequential filtering.
Horizontal Tracking	LSTM Neural Network	Mapped horizontal acceleration directly to camera tracking data to avoid exponential drift.

Algorithms & Data Processing

- **Missing Data Handling:** Missing data points caused by sensor transmission drops were handled by utilizing intersecting timeframes and applying linear interpolation.
- **Synchronization[3]:** Data synchronization between the IMU sensor and the camera was achieved using Fourier Transforms.

[2] https://github.com/JeongjunBae/Exercise-correction-using-wearable-devices/blob/main/final_presentation_English.pdf
[3] https://github.com/JeongjunBae/Synchronize_Signals/blob/main/ReadMe-EN.md

- **Computer Vision:** The pipeline sets a physical scale based on the barbell's size and precisely tracks the center of the weight plates.



- **Exercise Evaluation:** The algorithm detects signal peaks to compare maximum heights and reach times. It further evaluates the exercise cycle by measuring the time between peaks and calculates standard deviations to ensure cycle consistency.

Challenges

- Mapping acceleration directly to position yielded less-than-ideal results initially, likely due to the differing physical properties of the metrics.
- The third-party WL-Analysis app[4] failed to recognize several exercises, forcing us to quickly develop our own custom tracking program.

Future Work

- Design more robust algorithms to further minimize horizontal positional errors.
- Increase the system's capabilities beyond the currently supported exercises (Military Press, Squat, Barbell Row, and Deadlift).

Awards

- **First Prize:** 2021 Kyung Hee University Creative Design Competition.[5]
- **Second Prize:** 2021 Sungkyunkwan University (SKKU) Creative Design Competition.[6]

[4] https://play.google.com/store/apps/details?id=com.karolsmolak.wlanalysis&hl=en_US&pli=1
[5] https://drive.google.com/file/d/11RK3T1I0tvtgmYwRjneoGL2eivUWNryM/view?usp=drive_link
[6] https://drive.google.com/file/d/1lTSIbwsLu7ajZHV0UYx7xbmm7YR7AM7f/view?usp=drive_link